



PARADIGMAS DE PROGRAMACION

Batalla de Magos vs. Mortífagos

TP 2: GRUPO XI

INTEGRANTES

JUAN TIAGO PATRI | 39.766.474

JOAQUÍN CHINCHURRETA | 45.683.986

RAMIRO AGUSTÍN MEDINA | 45.400.641

IVANA RUIZ | 33.329.371

Docentes: Fabián Lagorio y Víctor Fernández





Introducción

El presente trabajo tiene como objetivo aplicar los principales conceptos del paradigma de la Programación Orientada a Objetos mediante el desarrollo de un sistema que simula una batalla entre dos facciones del universo de Harry Potter: los magos y los mortífagos. Durante el desarrollo se aplicaron principios de encapsulamiento, herencia, polimorfismo, abstracción y composición, complementados con la utilización de distintos patrones de diseño para lograr una solución flexible, mantenible y fácilmente extensible.

Además del desarrollo funcional del sistema, se implementó un conjunto de pruebas unitarias utilizando JUnit con el propósito de verificar el correcto funcionamiento de los componentes principales y asegurar la calidad del software.

Descripción general del proyecto

El sistema desarrollado representa un combate por turnos entre dos batallones enfrentados: un batallón de magos y otro de mortífagos. Cada personaje posee características propias, como nombre, nivel de magia, puntos de vida y un conjunto de hechizos que puede utilizar durante la batalla.

Cada ronda del combate se desarrolla de forma automática. Los personajes vivos de cada batallón ejecutan acciones siguiendo las reglas establecidas, pudiendo lanzar hechizos ofensivos, defensivos o de curación, además de utilizar objetos mágicos cuando las condiciones lo permiten.

El diseño fue realizado buscando minimizar el acoplamiento entre clases y facilitar futuras ampliaciones, permitiendo agregar nuevos personajes, hechizos, estados u objetos mágicos sin modificar la lógica principal del sistema.

Se implementaron los bonus:

- Objetos mágicos
- Personaje reactivo
- Hechizos con efectos prolongados

Descripción de las principales clases

Personaje

Es la clase abstracta principal del sistema. Contiene toda la información común a cualquier combatiente, como su nombre, nivel de magia, puntos de vida, estado actual, inventario de objetos mágicos y lista de hechizos conocidos.

También define las operaciones generales del combate, como lanzar hechizos, recibir daño, curarse, cambiar de estado, equipar objetos y aplicar efectos al comienzo de cada turno.

Todas las clases de personajes heredan de esta clase.

Mago

Clase abstracta que representa a los integrantes del bando de los magos. Hereda de Personaje y sirve como base para las distintas especializaciones.

Auror

Representa a los magos especializados en combate ofensivo.

Profesor

Representa personajes con mayor nivel de magia y mayor resistencia.

Estudiante

Representa los personajes con menor experiencia y un conjunto de hechizos más limitado.

Mortífago

Clase abstracta que representa al bando enemigo. También hereda de Personaje.

SeguidorComun

Representa a los mortífagos comunes.

Comandante

Además de ser un personaje, implementa el patrón Observer. Cuando un aliado es derrotado entra en un estado de furia que potencia automáticamente su siguiente ataque.

Hechizo

Es una interfaz que define el comportamiento común de todos los hechizos mediante el método ejecutar().

Cada hechizo implementa su propio comportamiento de forma independiente.

Los hechizos implementados son:

- Expelliarmus (daño directo).
- Expecto Patronum (curación).
- Protego (estado de resistencia).
- Petrificus Totalus (congelamiento).
- Avada Kedavra (envenenamiento).
- Desmaius (debilidad).

Gracias a esta estructura es posible agregar nuevos hechizos sin modificar los existentes.

EfectoEstado

Clase abstracta utilizada para representar el estado actual de un personaje.

Todos los efectos especiales modifican el comportamiento del personaje delegando las acciones al estado correspondiente.

Los estados implementados son:

- EstadoNormal
- EstadoDerrotado
- EstadoCongelado
- EstadoDebilidad
- EstadoResistencia
- EstadoVeneno

Los estados temporales heredan de la clase EfectoDurable, que administra automáticamente la duración de cada efecto.

GestorBatalla

Es la clase encargada de controlar toda la lógica del combate.

Entre sus responsabilidades se encuentran:

- administrar ambos batallones;
- controlar el desarrollo de las rondas;

- seleccionar las acciones de cada personaje;
- evitar la repetición de hechizos durante un turno;
- registrar el historial de hechizos lanzados;
- notificar los eventos importantes a los observadores.

También utiliza diferentes colecciones de Java:

- List para administrar secuencias.
- Map para almacenar el historial de hechizos por personaje.
- Set para impedir repetir hechizos durante un turno.

Batallón

Representa un equipo de combate.

Administra la colección de personajes pertenecientes al mismo bando y permite:

- agregar integrantes;
- obtener personajes vivos;
- obtener personajes derrotados;
- reincorporar personajes revividos.

También implementa la interfaz BatallaObserver.

LogBatalla

Implementa BatallaObserver y registra cronológicamente todos los eventos relevantes del combate, como los hechizos lanzados y las derrotas de personajes.

Reclutador

Centraliza la creación de personajes.

El usuario únicamente solicita un tipo de personaje y el Reclutador utiliza las factorías correspondientes para construir completamente el objeto con sus atributos y hechizos iniciales.

HechizoFactory

Centraliza la creación de los distintos hechizos del sistema evitando que otras clases conozcan los detalles de construcción.

Factorías de personajes

Cada tipo de personaje posee su propia factoría:

- AurorFactory
- ProfesorFactory
- EstudianteFactory
- ComandanteFactory
- SeguidorComunFactory

Estas clases son responsables de construir correctamente cada personaje asignándole su nivel de magia, puntos de vida máximos y hechizos iniciales.

Objetos mágicos

Como funcionalidad adicional se implementaron dos objetos mágicos:

CapaInvisibilidad

Permite que el personaje tenga una probabilidad de esquivar completamente un ataque recibido.

PiedraFilosofal

Permite revivir a un aliado derrotado restaurando parte de sus puntos de vida y reincorporándolo al combate.

Patrones de diseño utilizados

Strategy

El patrón Strategy se implementa mediante la interfaz Hechizo.

Cada hechizo representa una estrategia distinta para ejecutar una acción sobre un personaje. El GestorBatalla y los personajes únicamente conocen la interfaz común, sin depender de implementaciones concretas.

Este diseño permite incorporar nuevos hechizos sin modificar el resto del sistema.

State

El patrón State se implementa mediante la jerarquía encabezada por EfectoEstado.

Cada estado modifica el comportamiento del personaje frente a acciones como recibir daño, lanzar hechizos o curarse.

Los estados especiales (Veneno, Debilidad, Resistencia y Congelado) extienden EfectoDurable, permitiendo que los efectos desaparezcan automáticamente después de cierta cantidad de turnos.

De esta manera se evita utilizar grandes estructuras condicionales para controlar el comportamiento de cada personaje.

Factory Method

El patrón Factory Method se utiliza para centralizar la creación de personajes.

Las interfaces MagoFactory y MortifagoFactory definen el proceso general de construcción, mientras que cada factoría concreta crea un tipo específico de personaje.

El Reclutador actúa como punto único de acceso para la creación de personajes, ocultando completamente los detalles de construcción.

Asimismo, HechizoFactory centraliza la creación de los distintos hechizos del sistema.

Observer

El patrón Observer se implementa para notificar automáticamente los eventos importantes de la batalla.

La interfaz BatallaObserver define los eventos que pueden producirse.

Los observadores implementados son:

- LogBatalla, que registra los eventos.
- Batallón, que mantiene actualizado el estado del equipo.
- Comandante, que reacciona automáticamente cuando un aliado es derrotado entrando en estado de furia.

El GestorBatalla actúa como sujeto observable notificando todos estos eventos sin conocer el comportamiento específico de cada observador.

Conclusión

El proyecto permitió aplicar de manera integrada los principales conceptos de la Programación Orientada a Objetos desarrollando un sistema modular, reutilizable y fácilmente extensible.

La utilización de herencia y polimorfismo permitió representar distintos tipos de personajes compartiendo una misma estructura base, mientras que los patrones de diseño facilitaron la separación de responsabilidades y redujeron el acoplamiento entre componentes.

Asimismo, la implementación de objetos mágicos, personajes reactivos y efectos de estado prolongados demostró la flexibilidad alcanzada por la arquitectura desarrollada, permitiendo incorporar nuevas funcionalidades sin modificar significativamente el código existente.

Finalmente, el desarrollo de pruebas unitarias mediante JUnit permitió validar el correcto funcionamiento de las principales clases y garantizar un mayor nivel de confiabilidad del sistema construido.